División de microservicios

A continuación se detalla la división de los seis microservicios del núcleo de negocio de Conecta Nexus, seleccionando de forma estricta entre FastAPI y Django (Django REST Framework) para maximizar la eficiencia en tiempo de respuesta, desarrollo ágil y auditoría:

Microservicio	Framework	Justificación Técnica
auth	FastAPI	Enfoque asíncrono y ultraligero orientado al manejo continuo de credenciales seguras, firma criptográfica, emisión y revocación de tokens JWT. Garantiza latencias inferiores a 50 ms en validaciones cruzadas y esquemas de entrada estrictos con Pydantic.
profile	Django (DRF)	Dominio puramente relacional centrado en perfiles académicos y fichas corporativas. Su ORM maduro facilita el modelado de datos estructurados, validaciones de integridad referencial, inmutabilidad de código estudiantil y registro del consentimiento de la Ley 1581.
challenge	FastAPI	Diseñado para gestionar la carga y streaming asíncrono de documentos adjuntos técnicos (menores o iguales a 10 MB hacia Object Storage) y proveer consultas con filtrado reactivo de alta concurrencia en el catálogo público de problemáticas.
application	Django (DRF)	Requiere transaccionalidad atómica estricta (bloqueo atómico a «En Proceso», causal

Microservicio	Framework	Justificación Técnica
		obligatoria de rechazo y restricción de duplicidad RB-03 y RB-04). El soporte nativo de transacciones atómicas y bloqueos a nivel de fila de Django garantiza consistencia sin condiciones de carrera.
moderation	Django (DRF)	Aprovecha de forma estratégica el panel nativo de Django Admin para permitir al Administrador supervisar publicaciones, ejecutar suspensiones cautelares y auditar bitácoras inmutables sin requerir el desarrollo inmediato de interfaces dedicadas en frontend.
notification	FastAPI	Servicio desacoplado tipo Worker que escucha eventos de dominio asíncronos y despacha correos electrónicos transaccionales mediante tareas en segundo plano (Background Tasks) no bloqueantes, con bajo consumo de memoria y tolerancia a fallos SMTP.

Integrantes y responsabilidades

Asignación de equipos en parejas bajo el modelo de desarrollo guiado y Code Owners, garantizando corresponsabilidad técnica y revisión obligatoria en cada Pull Request:

Microservicio	Stack	Responsables Asignados (Code Owners)
auth	FastAPI	Anorak & Nelson

Microservicio	Stack	Responsables Asignados (Code Owners)
challenge	FastAPI	Anorak & Nelson
profile	Django	Anorak & Juan
application	Django	Anorak & Juan
moderation	Django	Juan & Nelson
notification	FastAPI	Juan & Nelson

Balance de Carga por Integrante

La distribución de responsabilidades es strictly simétrica y balanceada entre los tres desarrolladores, asignando exactamente cuatro microservicios por integrante con complementariedad de tecnologías (dos en FastAPI y dos en Django):

- **Anorak** (4 repositorios):
  - `auth` (FastAPI)
  - `challenge` (FastAPI)

    - `profile` (Django)
    - `application` (Django)
    - *Sinergia:* Concentra el núcleo de acceso, catálogo de retos y orquestación de la postulación de estudiantes.
- **Juan** (4 repositorios):
    - `profile` (Django)
    - `application` (Django)
    - `moderation` (Django)
    - `notification` (FastAPI)
    - *Sinergia:* Concentra la capa de datos de perfil, la lógica de matching y la gestión de auditoría y notificaciones derivadas de decisiones empresariales.
- **Nelson** (4 repositorios):
    - `auth` (FastAPI)
    - `challenge` (FastAPI)
    - `moderation` (Django)
    - `notification` (FastAPI)
    - *Sinergia:* Asegura la infraestructura de seguridad, el catálogo de necesidades y los extremos de supervisión y comunicación asíncrona.

Descripción de lo que se hará en lo que queda de semestre académico para hacer cada microservicio

A continuación se define la hoja de ruta técnica, los criterios de completitud funcional (Definition of Done) para dar por operativo cada componente y los mecanismos de comunicación inter-servicio:

1. auth (FastAPI)

- **Criterios para considerarse operativo:**
    - Endpoints REST funcionales para autorregistro de Estudiante y Empresa, inicio de sesión seguro y renovación/revocación de tokens JWT.
    - Almacenamiento de credenciales con hashing robusto (bcrypt/argon2) en la base de datos lógica dedicada (`db_auth`).
    - Exposición de endpoint de verificación de sesión (`/api/v1/auth/verify`) para consumo por el API Gateway.

    - Suite de pruebas unitarias con Pytest cubriendo casos de autenticación exitosa, credenciales inválidas y tokens expirados.

2. profile (Django)

- **Criterios para considerarse operativo:**
    - Modelos relacionales implementados para Estudiante (semestre validado entre 6.° y 10.°, habilidades, carrera) y Empresa (NIT, sector productivo, razón social).
    - Aplicación estricta de la regla de inmutabilidad del código de carrera estudiantil tras el registro.
    - Endpoint de registro y consulta de consentimiento informado de tratamiento de datos personales conforme a la Ley 1581.
    - Vistas serializadas de perfiles públicos restringidos que excluyen datos de contacto personal previo a la aceptación formal.
    - Base de datos lógica dedicada (`db_profiles`) con migraciones automáticas completas.

3. challenge (FastAPI)

- **Criterios para considerarse operativo:**
    - Ciclo de vida completo del reto modelado y controlado por estados: Borrador, Abierta, En Proceso, Cerrada y Cancelada.
    - Endpoints de publicación y actualización de problemáticas validados con Pydantic.
    - Servicio de carga, streaming y validación de archivos técnicos adjuntos menores o iguales a 10 MB interactuando con Object Storage (MinIO en local / Cloud Storage en nube).
    - Catálogo indexado con filtros multicriterio por área temática, sector productivo y habilidades requeridas.
    - Emisión del evento de dominio `ChallengePublished` hacia el bus de eventos cuando una problemática pasa a estado Abierta.
    - Base de datos lógica dedicada (`db_challenges`).

4. application (Django)

- **Criterios para considerarse operativo:**
    - Endpoint para formulación de postulación motivada por parte de estudiantes.
    - Regla de validación de negocio atómica RB-03: Bloqueo de postulaciones duplicadas sobre un mismo reto abierto.

    - Consulta síncrona hacia `profile` para verificar que el estudiante curse entre 6.° y 10.° semestre antes de admitir la postulación.
    - Transición transaccional segura ante decisión empresarial: si se Acepta, se cambia atómicamente el reto a «En Proceso» (invocando a `challenge`), se bloquean nuevas postulaciones y se revelan recíprocamente los canales de contacto (Ley 1581).
    - En caso de Rechazo, exigencia obligatoria del registro de causal justificada.
    - Emisión de eventos de dominio `ApplicationCreated`, `ApplicationAccepted` y `ApplicationRejected`.
    - Base de datos lógica dedicada (`db_applications`).

5. moderation (Django)

- **Criterios para considerarse operativo:**
    - Configuración personalizada de Django Admin con permisos exclusivos para el rol Administrador.
    - Modelos de auditoría institucional y bitácoras inmutables de supervisión (RNF03) que registran fecha, usuario y motivo de cada acción.
    - Endpoints y acciones administrativas para suspensión cautelar o baja definitiva de problemáticas que incumplan pertinencia académica o ética.
    - Comunicación síncrona o emisión de eventos hacia `challenge` y `auth` para ejecutar el bloqueo de contenidos o cuentas infractoras.
    - Base de datos lógica dedicada (`db_moderation`).

6. notification (FastAPI)

- **Criterios para considerarse operativo:**
    - Consumidor asíncrono escuchando los tópicos de eventos de negocio (`challenge-events` y `application-events`).
    - Servicio de despacho SMTP desacoplado configurado con Mailpit en entorno local y servidor transaccional en nube.
    - Plantillas de correo estructuradas para: notificación a empresa por nueva postulación recibida, notificación a estudiante con causal de rechazo, y notificación a ambas partes con datos de contacto tras aceptación formal.
    - Control de idempotencia y reintentos ante fallas temporales del servidor SMTP (RNF06).
    - Base de datos o almacenamiento ligero dedicado (`db_notifications` / Redis).

Mecanismos de Comunicación e Integración entre Microservicios

Para preservar el desacoplamiento arquitectónico y la autonomía tecnológica, los servicios interactuarán bajo dos patrones clave:

1. **Comunicación Síncrona (HTTP / REST OpenAPI):**
    - Canalizado a través de API Gateway (`/api/v1/*`), que valida el token JWT emitido por `auth` antes de enrutar la petición.
    - Llamadas internas punto a punto únicamente para validaciones críticas y atómicas (por ejemplo, `application` consultando semestre en `profile` o bloqueando el estado del reto en `challenge`).
2. **Comunicación Asíncrona (Event-Driven):**
    - Basada en bus de mensajería (Redis Pub/Sub en desarrollo local / Google Cloud Pub/Sub en despliegue).
    - Los servicios productores (`challenge` y `application`) publican eventos sin esperar la respuesta del receptor.
    - El servicio `notification` consume de forma desacoplada para garantizar que ninguna falla de red o de correo afecte la experiencia interactiva del usuario en el frontend.